



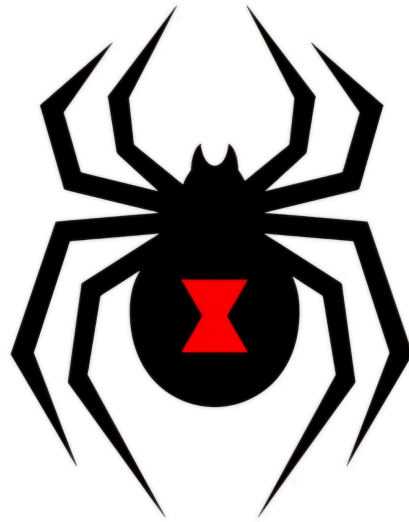
UNSA

Universidad Nacional de San Agustín

“AÑO DE LA ESPERANZA Y EL FORTALECIMIENTO DE LA DEMOCRACIA”

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



VIUDA NEGRA

Plan de pruebas de Aceptación

ASIGNATURA:

Pruebas de Software

DOCENTE:

Ing. Robert Edison Arisaca Mamani

INTEGRANTES:

Tejada Lazo Jordy Rolando (Test Lead)

Hurtado Bejarano Michael Steve (Test Analyst)

Jaita Chura Jose Manuel (Test Design)

Yare Chulunquia Kevin Pedro (Test Analyst)

Del Castillo Montoya Christopher Brad (Architect)

Índice

Índice	2
1. Introducción	4
1.1. Alcance	4
1.2. Referencias	4
1.3. Glosario	4
2. Contexto de las Pruebas	8
2.1. Proyecto / Subprocesos de Prueba	8
2.2. Elementos de Prueba	9
2.3. Alcance de la Prueba	10
2.4. Suposiciones y Restricciones	12
2.5. Partes Interesadas	12
3. Comunicación de las Pruebas	13
4. Registro de Riesgos	15
5. Estrategia de Prueba	18
5.1. Subprocesos de Prueba	18
5.1.1. Enfoque General — Caja Negra Manual	18
5.1.2. SP-1: Gestión de Partes — Alta Prioridad	19
5.1.3. SP-2: Control de Stock — Alta Prioridad	19
5.1.4. SP-3 a SP-5: Órdenes — Alta Prioridad	19
5.1.5. SP-6 a SP-8: Módulos Secundarios — Media Prioridad	19
5.2. Técnicas de Diseño de Prueba	20
5.3. Entregables del Plan de Integración:	21
5.4. Criterio de Aceptación	22
5.5. Métricas de Prueba	23
5.6. Requisitos del Entorno de Pruebas	23
5.6.1. Ambiente de Pruebas	23
5.6.2. Herramientas de Prueba	24
5.8.1. Criterios de Suspensión	25
5.8.2. Criterios de Reanudación	25
6. Actividades y Estimados de Prueba	25
6.1. Definición de la Estructura General de Pruebas	25
7. Personal	27
7.1. Roles, Actividades y Responsabilidades	27
7.2. Necesidades de Contratación	28
7.3. Necesidades de Capacitación	28
8. Cronograma	28

Control de versiones

Versión	Autor(es)	Descripción	Fecha
2.0	Tejada Lazo, Jordy Rolando	Versión inicial del documento	01/07/26
2.1	Tejada Lazo, Jordy Rolando	Corrección Secc. 2.3: pruebas E2E manuales con Playwright añadidas al alcance incluido. Ejecución automatizada de tests Playwright del CI (frontend.yaml) movida explícitamente al alcance excluido, con justificación ISO/IEC/IEEE 29119 y restricción del curso (pruebas manuales).	Julio 2026

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 4
--	---------------------------------------

1. Introducción

1.1. Alcance

El presente documento constituye el Plan de Pruebas de Aceptación (PAT) del sistema InvenTree v1.3.0, un sistema open-source de gestión de inventario. Este plan se focaliza exclusivamente en el nivel de pruebas de aceptación/sistema, verificando que el producto cumple con los requisitos funcionales y criterios de aceptación desde la perspectiva del usuario final, sin acceso al código fuente (caja negra).

Siguiendo la clasificación del estándar ISO/IEC/IEEE 29119, las pruebas de aceptación son el nivel más alto de la pirámide de pruebas y validan que el sistema satisface las necesidades reales del negocio. En el contexto del curso, el equipo opera como evaluadores externos que verifican el sistema desplegado en un entorno controlado de QA, simulando el rol de usuarios finales (administradores de inventario, operadores de almacén y compradores).

1.2. Referencias

- Repositorio oficial: <https://github.com/inventree/InvenTree>
- Documentación oficial de usuario: <https://docs.inventree.org/en/latest/>
- Demo pública oficial: <https://demo.inventree.org> (usuario: admin / contraseña: inventree)
- ISO/IEC/IEEE 29119 — Estándar internacional para pruebas de software
- Plan de Pruebas Unitarias InvenTree — Hito 2 (documento del equipo)
- Plan de Pruebas de Integración InvenTree v2.1 — Hito 2 (documento del equipo)
- PLANTILLA-Plan_de_Pruebas (Curso Pruebas de Software 2025-A, UNSA-EPIS)
- InvenTree REST API Documentation: <https://docs.inventree.org/en/latest/api/api/>
- InvenTree User Guide: <https://docs.inventree.org/en/latest/start/>

1.3. Glosario

En este documento se utilizan los siguientes términos abreviados:

Término	Definición
UAT	User Acceptance Testing — Pruebas de Aceptación del Usuario: validan que el sistema cumple los requisitos del negocio
Caja Negra	Técnica de prueba que evalúa el comportamiento externo del sistema sin acceso al código fuente
PE	Partición de Equivalencia: divide las entradas en clases válidas e inválidas para reducir casos de prueba
AVL	Análisis de Valores Límite: prueba los valores en los extremos de los rangos de entrada válidos/inválidos
InvenTree	Sistema open-source de gestión de inventario bajo licencia MIT. Backend Django/Python, Frontend React/TypeScript
Part	Entidad central de InvenTree que representa una pieza o componente del inventario
StockItem	Instancia física de una Part en una ubicación de almacén con cantidad y estado definidos
BOM	Bill of Materials (Lista de Materiales): define los componentes necesarios para fabricar un producto
PurchaseOrder	Orden de Compra: solicitud formal de adquisición de partes a un proveedor
SalesOrder	Orden de Venta: solicitud de entrega de partes a un cliente
BuildOrder	Orden de Fabricación: proceso de construcción de un producto a partir de su BOM
Plugin	Extensión funcional del sistema InvenTree que puede añadir o modificar comportamientos
Entorno QA	Ambiente de pruebas dedicado, separado del desarrollo, donde se ejecutan las pruebas de aceptación
Criterio de Aceptación	Condición que debe satisfacer el sistema para que una funcionalidad sea considerada aprobada

Término	Definición	Origen / Referencia en el proyecto
UAT	User Acceptance Testing — nivel de prueba que verifica que el sistema satisface los requisitos del negocio desde la perspectiva del usuario	ISO/IEC/IEEE 29119 — nivel 4 de la pirámide de pruebas; aplicado en este plan sobre la UI web de InvenTree

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 6
--	---------------------------------------

	final, sin acceso al código fuente	
Caja Negra	Técnica de diseño de pruebas que evalúa el comportamiento observable del sistema sin conocer ni acceder a su implementación interna	Base de todas las pruebas de este plan; el equipo actúa como usuario final sin leer el código fuente Django/React
Part	Entidad central del sistema InvenTree que representa una pieza, componente o producto del inventario. Tiene nombre, SKU, categoría, parámetros, imágenes y puede tener un BOM asociado	Modelo Part en src/backend/InvenTree/part/models.py
PartCategory	Categoría jerárquica que agrupa Parts relacionadas. Soporta anidamiento multinivel (padre → hijo → nieto)	Modelo PartCategory en part/models.py; visible en menú lateral de la UI
StockItem	Instancia física de una Part almacenada en una ubicación concreta con cantidad y estado definidos. Un mismo Part puede tener múltiples StockItems en distintas ubicaciones	Modelo StockItem en src/backend/InvenTree/stock/models.py
StockLocation	Ubicación física de almacén donde se guardan StockItems. Soporta jerarquía: Almacén → Pasillo → Estante → Casilla	Modelo StockLocation en stock/models.py
StockTracking	Registro histórico de todos los movimientos de un StockItem: transferencias, ajustes, consumos, retornos	Modelo StockItemTracking en stock/models.py; visible en la pestaña "History" de cada StockItem
BOM	Bill of Materials (Lista de Materiales) — define los componentes y cantidades necesarias para fabricar o ensamblar un Part	Modelo BomItem en part/models.py; gestionado desde la pestaña "BOM" de cada Part en la UI
BomItem	Línea individual dentro de un BOM que relaciona un Part padre con un Part componente con su cantidad	Modelo BomItem en part/models.py; referenciado en build/tests/test_api.py

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 7
--	---------------------------------------

	requerida y tolerancia	
PurchaseOrder	Orden de Compra — documento formal que registra la adquisición de Parts a un proveedor. Transiciona entre estados: Pending → Placed → Complete / Cancelled	Modelo PurchaseOrder en src/backend/InvenTree/order/models.py
PurchaseOrderLineItem	Línea individual de una PurchaseOrder que especifica el Part a comprar, cantidad, precio unitario y proveedor	Modelo PurchaseOrderLineItem en order/models.py
SalesOrder	Orden de Venta — documento que registra la entrega de Parts a un cliente. Estados: Pending → In Progress → Shipped → Complete / Cancelled	Modelo SalesOrder en order/models.py
ReturnOrder	Orden de Retorno — permite gestionar devoluciones de Parts desde un cliente. Añadido en versiones recientes de InvenTree	Modelo ReturnOrder en order/models.py; cubierto en order/tests/test_api.py
BuildOrder	Orden de Fabricación — proceso de construir un Part a partir de los componentes definidos en su BOM. Consume StockItems como materiales	Modelo Build en src/backend/InvenTree/build/models.py
BuildLine	Línea individual de una BuildOrder que representa la asignación de un componente del BOM al proceso de fabricación	Modelo BuildLine en build/models.py; referenciado en build/tests/test_api.py
Company	Entidad que representa a un proveedor, cliente o fabricante dentro del sistema. Una misma empresa puede tener múltiples roles simultáneamente	Modelo Company en src/backend/InvenTree/company/models.py
SupplierPart	Relación entre un Part de InvenTree y el Part equivalente en el catálogo de un proveedor específico,	Modelo SupplierPart en company/models.py

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 8
--	---------------------------------------

	con su SKU de proveedor y precio	
--	----------------------------------	--

2. Contexto de las Pruebas

2.1. Proyecto / Subprocesos de Prueba

InvenTree es un sistema open-source de gestión de inventario industrial que proporciona control de stock, seguimiento de partes, gestión de órdenes de compra/venta, fabricación (BOM), reportes y una API REST completa. El sistema se organiza en los siguientes módulos funcionales que constituyen los subprocesos de prueba de aceptación:

Repositorio GitHub	https://github.com/inventree/InvenTree (7.1k ★ · 1.4k forks · 17,657+ commits)
Versión analizada	master (rama principal, junio 2026)
Licencia	MIT License
Backend	Python 3.11 / 3.14 + Django 5.x + Django REST Framework
Frontend	React 18 + TypeScript + Mantine UI + TanStack Query + Vite
Bases de datos soportadas	SQLite (dev/test) · PostgreSQL 17 (producción) · MySQL 9 (producción)
Caché / Tareas async	Redis 8 + Django-Redis + Django Q2
CI/CD pipeline	GitHub Actions — qc_checks.yaml (691 líneas, 12+ jobs) + frontend.yaml
Fixtures de prueba verificadas	category · part · bom · stock · location · order · sales_order · transfer_order · return_order · build · company · supplier_part · users · settings
Cobertura objetivo (codecov.yml real)	Backend apps: 90% · Backend general: 92% · Frontend (web): 68% · Migrations: 40%

Cobertura mínima global (curso)	85% (el proyecto supera este umbral en el backend)
--	--

N°	Módulo	Funcionalidades Principales
SP-1	Gestión de Partes (Part)	Crear/editar/eliminar partes, categorías, parámetros, imágenes adjuntas, BOM
SP-2	Control de Stock	Gestionar ubicaciones, ajustar stock, transferir entre ubicaciones, registrar movimientos, conteos
SP-3	Órdenes de Compra (PurchaseOrder)	Crear/editar órdenes de compra, gestionar proveedores, recepcionar mercancía, states: Pending→Placed→Complete
SP-4	Órdenes de Venta (SalesOrder)	Crear/editar órdenes de venta, asignar stock, despachar, gestionar clientes
SP-5	Órdenes de Fabricación (BuildOrder)	Crear builds a partir del BOM, asignar materiales, completar fabricación, trackear producción
SP-6	Gestión de Empresas	Crear/editar proveedores, clientes, fabricantes; gestionar precios de proveedor
SP-7	Usuarios y Permisos	Crear usuarios, asignar grupos/roles, verificar acceso según permisos (Admin/View/Add/Change/Delete)
SP-8	Reportes y Etiquetas	Generar reportes PDF (stock, BOM, órdenes), imprimir etiquetas de partes y ubicaciones

2.2. Elementos de Prueba

Los elementos sujetos a prueba de aceptación son las funcionalidades accesibles desde la interfaz de usuario web de InvenTree, ejecutadas manualmente por el equipo en el entorno QA:

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 10
--	--

- Interfaz web React (Frontend): navegación, formularios de creación/edición, tablas de listado con filtros y búsqueda, modales de confirmación, dashboard principal
- Flujos de usuario completos end-to-end: desde la creación de una parte hasta su utilización en una orden de fabricación
- Control de acceso por roles: verificación de que cada rol (admin, user de solo lectura) accede únicamente a las funciones permitidas
- Validaciones de entrada en formularios: campos obligatorios, formatos, rangos numéricos, longitudes
- Mensajes de error y retroalimentación al usuario: claridad, exactitud y utilidad de los mensajes del sistema
- Consistencia de datos entre módulos: un ajuste de stock se refleja correctamente en los reportes y en las órdenes relacionadas

2.3. Alcance de la Prueba

Incluido en el alcance:

- Pruebas funcionales manuales de todos los módulos listados en la sección 2.1 (SP-1 a SP-8)
- Pruebas de caja negra aplicando técnicas de Partición de Equivalencia (PE), Análisis de Valores Límite (AVL), Tablas de Decisión y Transición de Estados
- Pruebas de usabilidad básica: navegación intuitiva, claridad de mensajes, consistencia visual
- Pruebas de compatibilidad de navegadores: Google Chrome (última versión), Mozilla Firefox (última versión)
- Verificación de permisos y control de acceso por rol de usuario
- Validación de flujos completos de negocio: ciclo completo Parte→Stock→Orden→Despacho
- Pruebas de transición de estados de órdenes (PurchaseOrder, SalesOrder, BuildOrder)
- Verificación de generación de reportes PDF básicos

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 11
---	---

- Pruebas E2E (End-to-End) de la interfaz web ejecutadas manualmente desde el navegador: el equipo simula flujos reales de usuario (login, navegación entre módulos, operaciones CRUD completas) verificando el comportamiento observable del sistema de extremo a extremo, sin acceso al código fuente. Este tipo de prueba es el que herramientas como Playwright automatizan, pero en este plan se ejecuta de forma manual por el equipo.

Excluido en el alcance:

- Pruebas unitarias de código fuente (cubiertas en el Hito 2 — Plan de Pruebas Unitarias)
- Pruebas de integración entre componentes internos via API REST (cubiertas en el Hito 2 — Plan de Pruebas de Integración)
- Pruebas de rendimiento, carga o estrés bajo concurrencia
- Pruebas de seguridad avanzadas (penetration testing, análisis de vulnerabilidades)
- Pruebas de la aplicación móvil companion de InvenTree
- Pruebas de plugins externos de terceros no incluidos por defecto en InvenTree
- Pruebas de infraestructura (Docker, servidor de producción, configuración de red)
- Pruebas de migración de base de datos entre versiones (cubiertas en el Plan de Integración)
- Pruebas de servicios externos: SMTP, OAuth, lectores de código de barras físicos
- Ejecución automatizada de los tests Playwright del pipeline CI de InvenTree (job frontend.yaml — Firefox y Chromium shards): estos tests automatizados pertenecen correctamente al nivel de pruebas de aceptación/sistema, pero su ejecución automatizada queda excluida de este plan dado que el curso exige pruebas manuales. Los mismos escenarios que cubren son ejecutados manualmente por el equipo tal como se

describe en el punto anterior (pruebas E2E manuales en el navegador).

2.4. Suposiciones y Restricciones

Suposiciones:

- El equipo utilizará la instancia demo pública <https://demo.inventree.org> como entorno de QA principal (usuario: admin / contraseña: inventree), que se reinicia periódicamente con datos frescos
- Alternativamente, se puede desplegar InvenTree localmente con 'invoke dev.server' e 'invoke dev.import-fixtures' para un entorno controlado
- Las pruebas funcionales se ejecutan manualmente, sin automatización, tal como indica el curso
- Se asume que InvenTree v1.3.0 es estable y las funcionalidades core están completas
- Todos los integrantes del equipo tienen acceso a la instancia de prueba con credenciales admin

Restricciones:

- Las pruebas son manuales (no automatizadas), lo que limita el volumen de casos ejecutables en el tiempo del sprint
- La instancia demo pública puede ser modificada por otros usuarios concurrentemente, lo que puede interferir con algunos casos de prueba — se recomienda el entorno local para pruebas destructivas
- El tiempo disponible limita la cobertura a los módulos de mayor prioridad para el negocio (SP-1 a SP-5)
- La corrección de defectos no está en el alcance del equipo (solo se reportan en GitHub Issues)

2.5. Partes Interesadas

Rol	Responsabilidades
Docente (Ing. Arisaca)	Aprobación de criterios de aceptación académicos, validación del plan, evaluación de entregables del Hito 3
Test Lead	Liderazgo del equipo, coordinación de actividades, aprobación de entregables, comunicación con docente, gestión de riesgos
Test Analyst (x2)	Ejecución de casos de prueba manuales, reporte de defectos en

	GitHub Issues, documentación de resultados
Test Architect	Configuración del entorno de pruebas QA, soporte técnico al equipo, análisis de defectos complejos
Test Design	Diseño detallado de casos de prueba, creación de datos de prueba, documentación técnica, elaboración de informes

3. Comunicación de las Pruebas

Esta sección define cómo se comunicará el equipo durante la ejecución de las pruebas de aceptación del Hito 3, asegurando que todos los miembros estén alineados con el avance, los defectos encontrados y las decisiones tomadas.

Protocolo de Comunicación

- Comunicación Interna: Se usará la red social (WhatsApp) para conversaciones rápidas, reuniones sincrónicas por Google Meet, y documentos colaborativos (Google Docs / Hoja de cálculo / Plataforma Clik Cup) para registro de acuerdos.
- Comunicación Externa: Se notificará al docente mediante los servicios de correo institucional o plataforma virtual de aprendizaje.
- Resolución de Conflictos: Se gestionará en primera instancia por el Test Lead. Si no se resuelve, se eleva al Docente.

Punto de Comunicación	Propósito	Frecuencia	Canal	Responsable	Audiencia
Reunión de inicio del Sprint	Distribuir casos de prueba, definir entorno y credenciales de QA	1 vez (inicio)	Google Meet / Presencial	Test Lead	Todo el equipo
Daily Stand-up	Estado de ejecución diaria, bloqueos, defectos nuevos	Diario (15 min)	WhatsApp	Test Analyst	Todo el equipo
Reporte de defecto	Registrar formalmente un bug encontrado	Al detectar	GitHub Issues	Quien lo detecta	Test Lead + Design

	durante la ejecución				
Revisión semanal con docente	Validar avance del sprint, recibir feedback académico	2 veces/semana	Presencial / Meet	Test Lead	Docente + Equipo
Retrospectiva de Sprint	Evaluar proceso, identificar mejoras para próximas iteraciones	Fin de sprint	Presencial / Meet	Test Lead	Todo el equipo
Informe final de resultados	Comunicar resultados oficiales al docente con evidencias	Fin del Hito 3	Documento .docx + Drive	Test Lead + Analyst	Docente
Reunión de inicio del Sprint	Distribuir casos de prueba, definir entorno y credenciales de QA	1 vez (inicio)	Google Meet / Presencial	Test Lead	Todo el equipo
Daily Stand-up	Estado de ejecución diaria, bloqueos, defectos nuevos	Diario (15 min)	WhatsApp	Test Analyst	Todo el equipo

Participantes del equipo:

1. Robert Edison Arisaca Mamani (Docente del curso)
2. Tejada Lazo, Jordy Rolando (Test Lead)
3. Hurtado Bejarano, Michael Steve (Test Analyst)
4. Yare Chullunquia, Kevin Pedro (Test Analyst)
5. Del Castillo Montoya, Brad Christopher (Test Architect)
6. Tejada Lazo, Jordy Rolando (Test Design)
7. Jaita Chura, Jose Manuel (Test Design)

Comentarios:

Todos los acuerdos relevantes se documentan en la carpeta compartida del equipo “Purebas_ViudaNegra” en el Drive. Se fomenta la comunicación asertiva y la colaboración proactiva.

4. Registro de Riesgos

En la siguiente tabla se identifican los riesgos del proyecto, así como se determina la severidad de cada uno de los riesgos multiplicando el impacto por la probabilidad de ocurrencia.

El impacto y la probabilidad se determinan teniendo en cuenta una escala de 1 al 5, donde 5 es el más alto.

N º	Riesgo identificado	Prob . (1-5)	Impac to (1-5)	Severid ad	Plan de Mitigación
1	La instancia demo pública es modificada por otros usuarios concurrentemente durante la ejecución	4	3	12	Usar entorno local ('invoke dev.server') para casos de prueba destructivos. Usar demo solo para lectura y creación.
2	Dificultad para instalar y configurar InvenTree localmente (dependencias, Docker)	3	4	12	Usar la demo oficial como entorno primario. Seguir la guía de la Wiki del equipo para instalación local.
3	Datos de prueba insuficientes o inconsistentes en el entorno QA para cubrir todos los escenarios	3	3	9	Ejecutar 'invoke dev.import-fixtures' para cargar datos de prueba estándar. Crear datos adicionales como parte de la ejecución.
4	Tiempo insuficiente para ejecutar todos los casos de prueba de los 8 módulos manualmente	3	4	12	Priorizar módulos SP-1 a SP-5 (alta criticidad). SP-6 a SP-8 como secundarios si hay tiempo disponible.

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 16
--	--

5	Comportamiento distinto entre la demo pública y una instalación local (versión, fixtures)	2	3	6	Documentar versión exacta de InvenTree usada en cada ejecución. Anotar las diferencias observadas.
6	Incompatibilidad de navegador con alguna funcionalidad del frontend React	2	3	6	Ejecutar todos los casos en Chrome primero. Si falla, verificar en Firefox y documentar diferencias.
7	Defecto bloqueante que impide continuar con la ejecución de un módulo completo	2	5	10	Registrar el defecto en GitHub Issues como BLOQUEANTE. Continuar con el siguiente módulo. Reportar al Test Lead inmediatamente.
8	Falta de comprensión de la funcionalidad de InvenTree para diseñar casos de prueba correctos	3	3	9	Leer la documentación de usuario de InvenTree (docs.inventree.org) antes de iniciar cada módulo. Usar la demo para familiarizarse.
9	Retraso de algún integrante en la ejecución de sus casos asignados	2	3	6	Revisión diaria en el tablero GitHub Projects. El Test Lead redistribuye carga si hay bloqueos.

Riesgos Técnicos

ID	Riesgo	P	I	Sev.	Mitigación / Contingencia
RT-01	El job 'postgres' falla en el fork por problema de conexión a la BD (INVENTREE_DB_HOST mal configurado, puerto no expuesto en el runner)	3	4	12	Verificar que las variables de entorno del fork coinciden exactamente con qc_checks.yaml; usar el template de .env del proyecto (contrib/env.sample)
RT-02	El job 'coverage' falla con error de compilación de traducciones	3	3	9	Los jobs de CI incluyen apt-dependency: gettext en su setup action. Localmente: apt-get install

	(compilemessages requiere gettext instalado)				gettext antes de ejecutar invoke dev.test --translations
RT-03	GitHub Actions del fork no ejecuta jobs por falta del secret CODECOV_TOKEN	4	2	8	El upload a Codecov fallará, pero los tests sí corren. Los jobs tienen continue-on-error para el upload de cobertura. El equipo puede verificar cobertura localmente con coverage report
RT-04	El job 'migration-tests' no se activa (requiere cambios en archivos de migración o label 'full-run')	4	3	12	Añadir el label 'full-run' a una PR en el fork para forzar la ejecución completa. O usar el workflow import_export.yaml que siempre corre el ciclo PG → SQLite
RT-05	El job 'chromium' de Playwright (4 shards) falla por timeout o recursos insuficientes en el runner gratuito	4	3	12	Ejecutar con --shard=1/4 solamente en primera instancia. El job 'firefox' (2 shards) es menos costoso. Ambos tienen timeout-minutes: 60
RT-06	Diferencias de comportamiento entre SQLite (local) y PostgreSQL (CI) causan que algunos tests pasen en uno y fallen en otro	3	4	12	El proyecto ya está diseñado para soportar los 3 motores; la mayoría de diferencias ya están corregidas. Documentar cualquier discrepancia encontrada.
RT-07	El task runner (invoke) no está disponible en el entorno local del equipo	4	3	12	pip install invoke (es una dependencia única). Todo el setup se hace vía invoke dev.setup-dev. Alternativamente, usar el devcontainer del proyecto (.devcontainer/) con Docker.

RT-08	Node.js versión incorrecta para los jobs del frontend (requiere v24, según env.node_version en los workflows)	3	3	9	Instalar nvm y usar nvm use 24. El setup action del proyecto gestiona esto automáticamente en CI.
RT-09	El job 'python' requiere que el servidor InvenTree esté corriendo en background; puede haber conflictos de puertos	3	3	9	El workflow usa el puerto 12345 (no el 8000 por defecto) para evitar conflictos. Localmente verificar que el puerto esté libre.
RT-10	Retraso en la ejecución de tests por parte de algún integrante del equipo	3	3	9	Asignación clara de jobs en la matriz RACI. El Test Architect ejecuta los jobs críticos (coverage + postgres) como mínimo para tener resultados de referencia.

5. Estrategia de Prueba

5.1. Subprocesos de Prueba

La estrategia de pruebas de aceptación de InvenTree se organiza en torno a 8 subprocesos funcionales (SP-1 a SP-8) descritos en la sección 2.1. Para cada subproceso se aplica el siguiente enfoque:

5.1.1. Enfoque General — Caja Negra Manual

Todas las pruebas de aceptación se ejecutan de forma MANUAL, sin automatización, desde la interfaz de usuario web de InvenTree. El equipo actúa como usuarios finales del sistema, evaluando el comportamiento observable sin acceso al código fuente. Cada caso de prueba documenta: condición previa, pasos a ejecutar, datos de entrada, resultado esperado y resultado obtenido.

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 19
--	--

5.1.2. SP-1: Gestión de Partes — Alta Prioridad

- Crear partes con datos válidos, inválidos y en límites de campo (nombre vacío, nombre de 100+ caracteres, SKU duplicado)
- Crear categorías jerárquicas y verificar que la navegación muestra correctamente la jerarquía
- Agregar parámetros personalizados a una parte y verificar que se guardan y muestran correctamente
- Crear un BOM (Lista de Materiales) con múltiples componentes y verificar cantidades
- Adjuntar imagen a una parte y verificar que se muestra en el detalle

5.1.3. SP-2: Control de Stock — Alta Prioridad

- Crear ubicaciones de almacén jerárquicas (almacén → pasillo → estante → casilla)
- Agregar items de stock a una parte con diferentes cantidades (0, 1, negativo — valor límite)
- Transferir stock entre ubicaciones y verificar que los totales se actualizan
- Realizar ajuste de inventario (agregar/quitar stock) y verificar el historial de movimientos
- Verificar que el stock muestra estado correcto: En Stock / Agotado / Reservado

5.1.4. SP-3 a SP-5: Órdenes — Alta Prioridad

- Crear una Orden de Compra y seguir el flujo completo: Pending → Placed → Complete (recepción de mercancía)
- Crear una Orden de Venta, asignar stock y completar el despacho
- Crear una Orden de Fabricación basada en un BOM y verificar el consumo de materiales
- Verificar que los estados de las órdenes transicionan correctamente y que no se pueden hacer transiciones inválidas

5.1.5. SP-6 a SP-8: Módulos Secundarios — Media Prioridad

- Crear un proveedor/cliente/fabricante y asociarlo a partes y órdenes
- Crear un usuario con rol limitado y verificar que no puede acceder a funciones de administración

- Generar un reporte PDF de stock y verificar que se descarga correctamente

5.2. Técnicas de Diseño de Prueba

El archivo `.github/workflows/qc_checks.yaml` (691 líneas, analizado completamente) define el siguiente pipeline. Cada job corresponde a un nivel de integración:

Técnica	Aplicación en InvenTree	Ejemplo concreto	Técnica	Aplicación en InvenTree
Partición de Equivalencia (PE)	Divide las entradas en clases válidas e inválidas. Reduce el número de casos necesarios.	Campo 'Nombre de Parte': válido ('Resistencia 10k'), inválido (vacío), inválido (solo espacios), inválido (>100 chars)	Partición de Equivalencia (PE)	Divide las entradas en clases válidas e inválidas. Reduce el número de casos necesarios.
Análisis de Valores Límite (AVL)	Prueba los valores exactamente en los límites de los rangos válidos/inválidos.	Cantidad de stock: 0 (límite), 1 (mínimo válido), 999999 (máximo), 1000000 (fuera de rango)	Análisis de Valores Límite (AVL)	Prueba los valores exactamente en los límites de los rangos válidos/inválidos.
Tablas de Decisión	Combina múltiples condiciones de entrada para identificar las reglas de negocio.	Permisos: Admin+módulo habilitado → Acceso. User+solo lectura → Ver. User+sin permiso → HTTP 403.	Tablas de Decisión	Combina múltiples condiciones de entrada para identificar las reglas de negocio.

Transición de Estados	Verifica que las entidades transicionan correctamente entre estados y que las transiciones inválidas son bloqueadas.	PurchaseOrder: Pending→Placed OK; Complete→Pending NO (inválido); Cancelled no puede ir a Complete.	Transición de Estados	Verifica que las entidades transicionan correctamente entre estados y que las transiciones inválidas son bloqueadas.
Casos de Uso / Escenarios	Simula flujos reales de negocio completos de un usuario final.	Flujo completo: Crear Parte → Agregar a BOM → Crear Build Order → Consumir Stock → Completar Fabricación.	Casos de Uso / Escenarios	Simula flujos reales de negocio completos de un usuario final.
Pruebas Exploratorias	Ejecución libre sin script previo para detectar errores no previstos en el diseño formal.	Navegar entre módulos libremente, combinar acciones inesperadas, probar funciones poco usadas.	Pruebas Exploratorias	Ejecución libre sin script previo para detectar errores no previstos en el diseño formal.

5.3. Entregables del Plan de Integración:

Entregable	Formato	Responsable	Entregable
------------	---------	-------------	------------

Plan de Pruebas de Aceptación (este documento)	.docx	Test Lead + Test Design	Plan de Pruebas de Aceptación (este documento)
Documento de Diseño de Casos de Prueba Funcionales	.docx	Test Design + Test Analyst	Documento de Diseño de Casos de Prueba Funcionales
Informe de Ejecución de Casos de Prueba Funcionales	.docx	Test Analyst + Test Lead	Informe de Ejecución de Casos de Prueba Funcionales
Registro de Defectos (GitHub Issues)	GitHub Issues	Todos los analistas	Registro de Defectos (GitHub Issues)
Evidencias de ejecución (capturas de pantalla)	PNG/JPEG + Drive	Quien ejecuta cada caso	Evidencias de ejecución (capturas de pantalla)
GitHub Wiki: Resultados del Sprint de Aceptación	Markdown	Test Design	GitHub Wiki: Resultados del Sprint de Aceptación
GitHub Projects: tablero actualizado al cierre del sprint	GitHub Projects	Test Lead	GitHub Projects: tablero actualizado al cierre del sprint

5.4. Criterio de Aceptación

- 100% de los casos de prueba de prioridad ALTA ejecutados (SP-1, SP-2, SP-3, SP-4, SP-5)
- Al menos 85% de los casos de prioridad MEDIA ejecutados (SP-6, SP-7, SP-8)
- Todos los defectos de severidad CRÍTICA y ALTA registrados en GitHub Issues con descripción, pasos de reproducción y evidencias
- No existen defectos críticos sin registrar
- Informe de Ejecución de Casos de Prueba Funcionales completado y revisado por el Test Lead
- GitHub Wiki actualizado con resultados del sprint
- Test Lead aprueba formalmente el cierre del sprint de aceptación

5.5. Métricas de Prueba

Métrica	Descripción / Fórmula	Métrica
Total de casos diseñados	Número total de casos de prueba en el Documento de Diseño	Total de casos diseñados
Casos ejecutados (%)	$(\text{Casos ejecutados} / \text{Total diseñados}) \times 100\%$	Casos ejecutados (%)
Tasa de éxito	$(\text{Casos PASS} / \text{Total ejecutados}) \times 100\%$ — meta: $\geq 90\%$	Tasa de éxito
Tasa de fallo	$(\text{Casos FAIL} / \text{Total ejecutados}) \times 100\%$	Tasa de fallo
Defectos detectados	Número total de GitHub Issues creados durante el sprint de aceptación	Defectos detectados
Defectos por severidad	Distribución: Crítico / Alto / Medio / Bajo	Defectos por severidad
Cobertura de requisitos	$(\text{Módulos probados} / \text{Total módulos}) \times 100\%$	Cobertura de requisitos

5.6. Requisitos del Entorno de Pruebas

5.6.1. Ambiente de Pruebas

Componente	Especificación
Entorno QA primario	https://demo.inventree.org — instancia oficial mantenida por el proyecto InvenTree (usuario: admin / pass: inventree)
Entorno QA alternativo	InvenTree local: 'invoke dev.server' en localhost:8000 con datos de fixtures cargados vía 'invoke dev.import-fixtures'
Navegadores	Google Chrome (última versión estable) — Principal. Mozilla Firefox (última versión) — Secundario para verificación cruzada
Sistema Operativo	Windows 10/11 (mínimo 1 miembro), Ubuntu 22.04+ compatible

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 24
--	--

Resolución de pantalla	Mínimo 1366×768 para que la interfaz React de InvenTree se muestre correctamente
Conexión a internet	Requerida para usar la instancia demo pública. No requerida para entorno local.
Herramientas de captura	Snipping Tool (Windows), Flameshot (Linux) para evidencias de ejecución

5.6.2. Herramientas de Prueba

Herramienta	Versión	Función en las pruebas de aceptación
InvenTree Web UI (React)	v1.3.0	Interfaz principal sobre la que se ejecutan todas las pruebas manuales
Google Chrome	Última estable	Navegador principal para ejecución de pruebas funcionales manuales
Mozilla Firefox	Última estable	Navegador secundario para verificación de compatibilidad cruzada
GitHub Issues	N/A	Registro formal de defectos encontrados durante la ejecución de pruebas
GitHub Projects	N/A	Tablero Scrum/Kanban para seguimiento de tareas del sprint de aceptación
GitHub Wiki	N/A	Documentación del proceso de pruebas y resultados del equipo
Google Drive / Docs	N/A	Almacenamiento de evidencias (capturas), documentos de casos y resultados
Microsoft Word / LibreOffice	N/A	Elaboración de informes y documentos entregables del Hito 3

5.7. Re-testing y Regresión de las Pruebas

Dado que las pruebas de aceptación son manuales y el equipo no corrige defectos (solo los reporta), la estrategia de re-testing y regresión es la siguiente:

- Re-testing: Si se detecta que un defecto reportado en un sprint anterior fue corregido por el equipo de desarrollo de InvenTree (commit en el

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 25
---	---

repositorio oficial), se re-ejecuta el caso de prueba correspondiente en la nueva versión

- Regresión parcial: Al inicio de cada ciclo de ejecución, se re-ejecuta una selección de los 10 casos más críticos de módulos ya probados para verificar que no hubo regresión
- Regresión completa: Solo se ejecuta si se detecta un cambio de versión significativo de InvenTree entre el inicio y el cierre del sprint
- Ciclos de prueba: Se contemplan mínimo 2 ciclos de ejecución — uno inicial de exploración y uno de confirmación/regresión

5.8. Criterios de Suspensión y Reanudación

5.8.1. Criterios de Suspensión

- La instancia de InvenTree no es accesible (demo caída, fallo en servidor local) por más de 2 horas
- Se detecta un defecto bloqueante que impide ejecutar más del 40% de los casos pendientes de un módulo
- Los datos del entorno de prueba están corruptos y no pueden recuperarse con 'invoke dev.import-fixtures'
- Menos de 2 integrantes del equipo están disponibles para continuar la ejecución

5.8.2. Criterios de Reanudación

- El entorno de pruebas fue restaurado y verificado por el Test Architect
- Los defectos bloqueantes fueron registrados y se continuará con módulos alternativos
- Al menos 3 integrantes del equipo están disponibles para retomar la ejecución
- El Test Lead aprueba formalmente la reanudación mediante comunicación al equipo

6. Actividades y Estimados de Prueba

6.1. Definición de la Estructura General de Pruebas

El Sprint 1 del Hito 3 se focaliza en el diseño y ejecución manual de pruebas de aceptación de los módulos principales de InvenTree. Las actividades se distribuyen en las siguientes fases:

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 26
--	--

N°	Actividad	Responsable	Hrs. Est.	Descripción / Entregable
1	Configuración y verificación del entorno QA	Test Architect	3 hrs	Acceder a demo.inventree.org y/o desplegar local. Verificar login admin, carga de fixtures, navegación básica funcional.
2	Lectura de documentación de usuario InvenTree	Todo el equipo	4 hrs	Leer docs.inventree.org (partes, stock, órdenes) para entender el comportamiento esperado antes de diseñar/ejecutar.
3	Diseño de Casos de Prueba Funcional (SP-1 a SP-5)	Test Design + Analyst	8 hrs	Diseñar mínimo 40 casos de prueba aplicando PE, AVL, Tablas de Decisión y Transición de Estados. Entregable: Documento de Diseño.
4	Diseño de Casos de Prueba Funcional (SP-6 a SP-8)	Test Design	4 hrs	Diseñar mínimo 15 casos adicionales para módulos secundarios (Empresas, Usuarios, Reportes).
5	Primer ciclo de ejecución (SP-1 y SP-2)	Test Analyst x2	6 hrs	Ejecutar manualmente todos los casos de Gestión de Partes y Control de Stock. Registrar resultado (PASS/FAIL) y evidencias.
6	Primer ciclo de ejecución (SP-3, SP-4 y SP-5)	Test Analyst x2	6 hrs	Ejecutar casos de Órdenes de Compra, Venta y Fabricación. Registrar resultado y evidencias.
7	Ejecución módulos secundarios (SP-6, SP-7, SP-8)	Test Analyst + Design	4 hrs	Ejecutar casos de Empresas, Usuarios/Permisos y Reportes. Documentar resultados.
8	Registro de defectos en GitHub Issues	Todo el equipo	3 hrs	Para cada caso FAIL: crear GitHub Issue con título, descripción, pasos de reproducción, resultado esperado vs obtenido, captura de pantalla y severidad.
9	Segundo ciclo: regresión y re-ejecución de casos FAIL	Test Analyst	3 hrs	Re-ejecutar los 10 casos más críticos y los que fallaron en el primer ciclo. Verificar si persisten o se resolvieron.
10	Elaboración del Informe de Ejecución de Pruebas	Test Lead + Analyst	5 hrs	Redactar informe formal con: resumen ejecutivo, tabla de resultados por módulo, defectos encontrados, métricas, conclusiones.
11	Actualización GitHub Wiki y GitHub Projects	Test Design	2 hrs	Publicar resultados en la Wiki. Cerrar los Issues completados en el tablero. Actualizar el dashboard del sprint.
12	Revisión final con docente y preparación de sustentación	Test Lead	2 hrs	Presentar resultados al docente. Recibir feedback. Ajustar el informe si es necesario. Preparar demo del sistema.

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 27
--	--

	TOTAL ESTIMADO		50 hrs	Distribuido entre 5 integrantes (~10 hrs por persona durante el Hito 3)
--	-----------------------	--	---------------	---

7. Personal

Esta sección describe los recursos humanos necesarios para llevar a cabo el esfuerzo de pruebas. Se especifican los roles, responsabilidades, necesidades de contratación y los entrenamientos requeridos.

7.1. Roles, Actividades y Responsabilidades

Para asegurar un proceso de pruebas bien organizado, se definen claramente los roles y responsabilidades del personal involucrado. Se utiliza una matriz RACI para indicar quién es Responsable (R), Aprobador/Responsable final (A), Consultado (C) e Informado (I) en cada actividad clave del proceso de pruebas donde:

- **R – Responsable:** Las personas que ejecutan la tarea o actividad, es decir, son quienes hacen el trabajo.
- **A – Aprobador / Responsable final (Accountable):** La persona que tiene la autoridad final sobre la actividad y se asegura de que se complete correctamente. Solo puede haber un “A” por actividad.
- **C – Consultado:** Personas que deben ser consultadas antes o durante la ejecución de la actividad. Son expertos o partes clave que brindan asesoría o información.
- **I – Informado:** Personas que deben ser notificadas del avance o resultados de la actividad. No participan directamente, pero necesitan estar al tanto.

Actividad	Docente	Test Lead	Test Analyst	Test Architect	Test Design
Configuración del entorno QA	I	A	C	R	I
Diseño de casos de prueba	I	A	C	I	R
Ejecución manual de pruebas	I	A	R	C	C
Registro de defectos (GitHub Issues)	I	A	R	C	R

Elaboración del Informe de Ejecución	I	R	C	I	R
Actualización GitHub Wiki y Projects	I	A	I	I	R
Aprobación y cierre del sprint	A	R	I	I	I

7.2. Necesidades de Contratación

En base al análisis de la carga de trabajo estimada y el cronograma definido para las actividades de prueba del sistema TEAMMATES, se ha llegado a la conclusión de que el equipo de pruebas funcionales cuenta con los integrantes necesarios para ejecutar el plan de pruebas de manera efectiva.

7.3. Necesidades de Capacitación

- Se requiere una sesión de inducción técnica de 1-2 horas para todos los integrantes del equipo antes de iniciar la ejecución, cubriendo los siguientes temas:
- Navegación básica por la interfaz web de InvenTree: menús, módulos, formularios principales
- Flujo completo del sistema: desde la creación de una Part hasta su uso en una BuildOrder
- Acceso y uso del entorno QA (demo.inventree.org o instancia local)
- Cómo leer y ejecutar un caso de prueba del Documento de Diseño
- Cómo registrar un defecto correctamente en GitHub Issues (formato, severidad, evidencias)
- Cómo actualizar el GitHub Projects con el estado de las tareas asignadas

8. Cronograma

El Sprint 1 del Hito 3 abarca las semanas posteriores a la entrega del Hito 2 (10 de junio de 2026). La presentación oficial del Hito 3 se coordina con el docente.

<i>Actividad</i>	<i>Responsable</i>	<i>Inicio</i>	<i>Fin</i>
Configuración entorno QA	Test Architect	Semana 1	Semana 1

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 29
--	--

Lectura documentaci ón InvenTree	Todo el equipo	Semana 1	Semana 1
Diseño de casos SP-1 a SP-5	Test Design + Analyst	Semana 1	Semana 2
Diseño de casos SP-6 a SP-8	Test Design	Semana 2	Semana 2
1er ciclo ejecución SP-1 y SP-2	Test Analyst ×2	Semana 2	Semana 2